1.

在单模式串匹配这一问题下, KMP 给出的 `next` 描述了模式前缀失配时下一个尝试匹配的前缀, **不能保证跳转到的前缀能够成功匹配**, 可能发生若干次跳转; 而 DFA 的转移函数 δ 描述了当前匹配到某前缀 (在某个状态), 再增加一个字符 a 时, 新的匹配前缀位置 (新的状态), **能够保证新状态代表的前缀成功匹配**, 仅需进行一次跳转.

KMP 跳转 `next` 树的过程可以写作:

```
// P[1..]: 模式串
int trans(int q, char a) {
    while (q && P[q + 1] != a) q = next[q];
    if (P[q + 1] == a) q = next[q];
    return q;
}
```

这就是对 DFA 的转移函数的实现.

2.

假定字符集大小 $|\Sigma|$ 为常数, 输入字符串 S 下标从 1 起.

$$
\begin{array}{l}
last = \mathrm{dict}\{(c, 0) : c \in \Sigma\} \\
(p, \ell, s) \leftarrow (0, 0, 0) \\
\textbf{for } \ i = 1 \ \textbf{ to } \ n \ \textbf{ do} \\
\quad\quad p \leftarrow \max(p, last[S[i]]) \\
\quad\quad \textbf{if } \ \ell < i - p \ \textbf{ then} \\
\quad\quad\quad\quad (\ell, s) \leftarrow (i - p, p + 1) \\
\quad\quad last[S[i]] \leftarrow i \\
\textbf{output } \ (S[s : s + \ell - 1], s)
\end{array}
$$

3.

考虑每次在结果字符串末尾加入一个字符, 这时:

- 若新字符为 `b`, 立即删除之;
- 若结果字符串末尾为 `a` 且新字符为 `c`, 原字符串删除末尾.

因此, 用指针 i 维护结果字符串末尾, 用指针 j 扫描原字符串加入字符, 就能模拟完成操作.

仍假设字符串下标从 1 起 (从 0 起也是平凡的).

```
function modify(S) → int :
    (n, i) ← (length(S), 0)
    for j = 1 to n do
        c ← S[j]
        if c = b then continue
        elif c ≠ c ∨ i = 0 ∨ S[i] ≠ a then
            (S[i], i) ← (c, i + 1)
        else
            i ← i − 1
    return i
```

复杂度明显是 $\mathcal{O}(n)$.

4.

若 $fp[k] = i$, 则 $T[: i + k - 1]$ 能匹配的 P 的最长前缀一定是 $P[: k - 1]$. 若更短, 则 $T[i : i + k - 1] \neq P[: k - 1]$; 若更长, 与 fp 的最小性矛盾. 因此可以执行 KMP 的匹配过程, 在每次完成前缀匹配后更新 fp 数组即可.

不妨简记 `first_pos` 为 `fp`.

```
next ← kmp(P)
fp ← [−1]^m
j ← −1
for i = 0 to n − 1 do
    while j ≥ 0 ∧ P[j + 1] ≠ T[i] do
        j = next[j]
    j ← j + [P[j + 1] = T[i]]
    if j ≥ 0 ∧ fp[j + 1] = −1 then
        fp[j + 1] ← i − j
```